

TECHNICAL ARTICLE

PostgreSQL VACUUM

Why It Exists, What Happens When It Falls Behind, and How to Recover Safely

Validated against PostgreSQL 18 official documentation

Prepared as an operations-focused maintenance reference | September 2026

Scope Covers MVCC cleanup, dead tuples, reusable space, visibility map, autovacuum thresholds, bloat, freeze protection, monitoring, blockers, troubleshooting, tuning, recovery, and production guardrails. Test changes with representative workload and deployed-version documentation.

Table of Contents

1. Executive Summary
2. Why PostgreSQL Needs VACUUM
3. MVCC, Dead Tuples, and Reusable Space
4. What Standard VACUUM Actually Does
5. Autovacuum Trigger Logic
6. What Happens When VACUUM Falls Behind
7. Monitoring and Diagnostic Scripts
8. Root-Cause Analysis
9. Troubleshooting Workflow
10. Recovery and Fix Procedures
11. Autovacuum Tuning
12. VACUUM Options and Special Cases
13. Prevention and Production Hardening
14. Common Mistakes and Corrections
15. Production Checklist
16. Official-Documentation Validation
17. References

1. Executive Summary

PostgreSQL uses MVCC to let readers and writers work concurrently. UPDATE normally creates a new tuple version, and DELETE makes an old version obsolete. Those obsolete tuples cannot be removed immediately because an older transaction might still need them. VACUUM later determines which versions are no longer visible to any relevant snapshot and makes their space reusable.

Key Note: Standard VACUUM usually does not shrink the table file or return all space to the operating system. It makes space reusable inside the relation, maintains visibility and free-space information, removes dead index entries where appropriate, and freezes old tuple metadata.

When vacuum falls behind, tables and indexes can grow, scans and cache efficiency can worsen, index-only scans may perform more heap fetches, planner statistics can become stale if ANALYZE also lags, and old XIDs can approach wraparound protection limits. The correct response begins with evidence and blockers, not an automatic VACUUM FULL.

2. Why PostgreSQL Needs VACUUM

Responsibility	Why it matters
Reuse dead-tuple space	Prevents unbounded heap growth during UPDATE/DELETE churn.
Clean index references	Removes dead index tuples when index cleanup is performed.
Update visibility map	Enables page skipping and supports index-only scans.
Maintain free space map	Helps future inserts find reusable page space.
Freeze old XIDs/MXIDs	Prevents transaction and MultiXact wraparound failures.
Update relation statistics	VACUUM updates some pg_class counts; VACUUM ANALYZE separately refreshes planner distributions.
Truncate empty tail pages	Plain VACUUM may return contiguous empty pages at relation end when truncation succeeds.

3. MVCC, Dead Tuples, and Reusable Space

```
CREATE TABLE vacuum_demo(id integer PRIMARY KEY, status text);
INSERT INTO vacuum_demo VALUES (1, 'new');
SELECT xmin, xmax, ctid, * FROM vacuum_demo;
UPDATE vacuum_demo SET status='processed' WHERE id=1;
SELECT xmin, xmax, ctid, * FROM vacuum_demo;
```

A tuple becomes dead only when no relevant transaction can see it. A long-running snapshot, prepared transaction, replication slot `xmin/catalog_xmin`, or standby feedback can hold back the cleanup horizon. `VACUUM` cannot safely remove a tuple that is still potentially visible.

Term	Meaning
Live tuple	Visible to current/future transactions under normal snapshot rules.
Recently dead	Obsolete but still potentially visible to an older snapshot.
Dead tuple	No longer needed and eligible for removal/reuse.
HOT update	Update that can avoid new index entries when indexed columns do not change and page space allows.
Free space	Heap space reusable by later tuples; not necessarily returned to OS.
Bloat	Allocated relation space that is inefficiently used; estimated, not identical to <code>n_dead_tup</code> .

4. What Standard VACUUM Actually Does

1. Scans heap pages, using the visibility map to skip pages when safe.
2. Identifies dead tuple item identifiers and coordinates index cleanup.
3. Removes dead line pointers after indexes no longer reference them.
4. Records free space and updates visibility/all-frozen information.
5. May truncate empty pages from the physical end of the heap.
6. Advances freeze metadata when all required pages are processed.
7. Runs concurrently with ordinary reads and writes without taking `ACCESS EXCLUSIVE` on the table.

```
VACUUM (VERBOSE, ANALYZE false) public.orders;
VACUUM (ANALYZE, VERBOSE) public.orders;
```

```
-- Database-wide within the connected database:
VACUUM (VERBOSE);
```

Transaction rule VACUUM cannot run inside a transaction block. VACUUM FULL is a table rewrite, requires ACCESS EXCLUSIVE, needs extra disk space, and is not routine maintenance.

5. Autovacuum Trigger Logic

PostgreSQL 18 uses separate thresholds for vacuuming obsolete tuples, insert-driven vacuuming, and analyzing changed rows. Counts are eventually consistent statistics and thresholds can be overridden per table.

```
vacuum threshold = least(
    autovacuum_vacuum_max_threshold,
    autovacuum_vacuum_threshold
    + autovacuum_vacuum_scale_factor * pg_class.reltuples
)

analyze threshold = autovacuum_analyze_threshold
    + autovacuum_analyze_scale_factor * pg_class.reltuples

SELECT name, setting, unit, source
FROM pg_settings
WHERE name LIKE 'autovacuum%'
      OR name IN ('track_counts', 'maintenance_work_mem',
                  'vacuum_buffer_usage_limit', 'log_autovacuum_min_duration')
ORDER BY name;
```

Trigger	Purpose
UPDATE/DELETE threshold	Vacuum after enough tuples become obsolete.
Insert threshold	Vacuum insert-heavy tables to improve visibility/freeze state.
Analyze threshold	Refresh planner statistics after enough row changes.
Freeze maximum age	Force anti-wraparound vacuum even if autovacuum is otherwise disabled.
Per-table reloptions	Tune large/hot tables independently of cluster defaults.

6. What Happens When VACUUM Falls Behind

Symptom	Mechanism	Impact
Growing heap	Dead space is not reused fast enough.	More pages, slower scans, larger backup/storage footprint.
Growing indexes	Dead index entries and churn accumulate.	More cache misses and index I/O.
More heap fetches	Visibility map has fewer all-visible pages.	Index-only scans lose benefit.
Stale optimizer view	ANALYZE may also lag.	Bad row estimates and plan changes.
Long vacuum cycles	More pages/dead items per run.	Workers remain busy; other relations wait.
Repeated index cycles	Dead-item memory fills during a large vacuum.	Extra index/heap passes and longer runtime.
Transaction horizon pinned	Old snapshot, slot, or prepared xact retains tuples.	VACUUM runs but cannot remove expected rows.
XID/MXID age rises	Freezing cannot keep pace.	Anti-wraparound work, warnings, eventual write outage protection.
Disk pressure	Relations and temporary/maintenance work consume capacity.	Performance degradation or operational failure.

Diagnostic distinction `n_dead_tup` is an estimate of dead tuples, not a byte-accurate bloat measurement. A table can be large because of reusable free space, row width, fillfactor, or historical growth. Confirm with relation-size trends and appropriate bloat inspection.

7. Monitoring and Diagnostic Scripts

7.1 Table maintenance status

```
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       n_ins_since_vacuum, n_mod_since_analyze,
       last_vacuum, last_autovacuum,
```

```

        last_analyze, last_autoanalyze,
        vacuum_count, autovacuum_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 50;

```

7.2 Relation and index size

```

SELECT c.oid::regclass AS relation,
       pg_size_pretty(pg_relation_size(c.oid)) AS heap,
       pg_size_pretty(pg_indexes_size(c.oid)) AS indexes,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total
FROM pg_class c
WHERE c.relkind IN ('r','m')
ORDER BY pg_total_relation_size(c.oid) DESC
LIMIT 50;

```

7.3 Running VACUUM progress

```

SELECT pid, datname, relid::regclass AS relation, phase,
       heap_blks_scanned, heap_blks_total, heap_blks_vacuumed,
       index_vacuum_count, dead_tuple_bytes, max_dead_tuple_bytes
FROM pg_stat_progress_vacuum
ORDER BY pid;

```

7.4 Old transactions and prepared transactions

```

SELECT pid, username, state, xact_start,
       age(backend_xid) AS xid_age,
       age(backend_xmin) AS xmin_age,
       left(query,200) AS query
FROM pg_stat_activity
WHERE backend_xid IS NOT NULL OR backend_xmin IS NOT NULL
ORDER BY greatest(age(backend_xid),age(backend_xmin)) DESC NULLS LAST;

SELECT gid, prepared, owner, database, age(transactionid) AS xid_age
FROM pg_prepared_xacts ORDER BY xid_age DESC;

```

7.5 Replication slots and XID age

```

SELECT slot_name, slot_type, active,
       age(xmin) AS xmin_age, age(catalog_xmin) AS catalog_xmin_age,
       restart_lsn, confirmed_flush_lsn
FROM pg_replication_slots
ORDER BY greatest(age(xmin),age(catalog_xmin)) DESC NULLS LAST;

SELECT datname, age(datfrozenxid) AS xid_age,
       mxid_age(datminmxid) AS mxid_age
FROM pg_database ORDER BY xid_age DESC;

```

8. Root-Cause Analysis

Root cause	Evidence	Corrective direction
------------	----------	----------------------

Root cause	Evidence	Corrective direction
Threshold too high for large table	Dead tuples grow substantially before trigger.	Lower per-table scale factor/threshold after measurement.
Workers saturated	All workers busy; eligible tables wait.	Tune worker capacity, cost/memory, and table scheduling.
Long transaction	Old xact_start/backend_xmin.	Application-approved commit, rollback, or termination.
Prepared transaction	Old pg_prepared_xacts entry.	Resolve business outcome with COMMIT/ROLLBACK PREPARED.
Replication slot	Old xmin/catalog_xmin or abandoned consumer.	Restore consumer or drop only verified unused slot; rebuild may be required.
Lock conflict	Autovacuum skipped/waiting; blocker in pg_locks/activity.	Correct DDL/maintenance conflict and scheduling.
I/O or disk pressure	Slow phases, high latency, limited free space.	Restore headroom and reduce competing I/O.
Insufficient dead-item memory	Multiple index_vacuum_count cycles.	Test autovacuum_work_mem/maintenance_work_mem carefully.
Autovacuum disabled per table	reloptions shows autovacuum_enabled=false.	Re-enable routine maintenance; anti-wraparound may still launch.
Insert-only workload	Few dead tuples but weak visibility/freeze progress.	Tune insert-triggered vacuum and freeze strategy.

9. Troubleshooting Workflow

8. Capture PostgreSQL version, table, symptoms, growth trend, errors, and recent changes.
9. Check disk space before starting heavy maintenance.
10. Confirm whether VACUUM/autovacuum is running and whether progress advances.
11. Compare n_dead_tup, size, last vacuum/analyze, and per-table reloptions.
12. Identify horizon blockers: open transactions, prepared xacts, slots, and standby feedback.
13. Check locks, worker saturation, I/O latency, and repeated index cycles.

14. Resolve only confirmed blockers with owner approval.
15. Run targeted plain VACUUM VERBOSE and monitor it.
16. Recheck dead tuples, size trend, visibility behavior, XID age, and query performance.
17. Tune per table based on observed churn and duration; avoid global guesswork.
18. Document cause, change, rollback, and preventive alerts.

10. Recovery and Fix Procedures

10.1 Standard catch-up

```
VACUUM (VERBOSE, ANALYZE) public.orders;

-- Command-line, multiple jobs across relations/databases as designed:
vacuumdb --dbname=appdb --analyze --verbose --jobs=4
```

10.2 Remove a confirmed horizon blocker

```
-- Only after application/business approval:
SELECT pg_terminate_backend(<pid>);

COMMIT PREPARED '<gid>';
-- or ROLLBACK PREPARED '<gid>';

-- Only for a verified abandoned slot:
SELECT pg_drop_replication_slot('<slot_name>');
```

10.3 Severe physical bloat

Method	Lock/space profile	Use
Plain VACUUM	Concurrent; usually no full shrink.	Routine cleanup and reuse.
VACUUM FULL	ACCESS EXCLUSIVE; rewrites; extra disk.	Planned shrink after major deletion when downtime is acceptable.
CLUSTER	ACCESS EXCLUSIVE rewrite; ordering benefit.	Planned physical reordering and shrink.
pg_repack extension	Additional setup/resources; lower blocking design.	Where approved and supported for online-style reorganization.

Do not jump to FULL VACUUM FULL is not a cure for autovacuum configuration or pinned horizons. It can cause downtime, needs extra space, and the table can bloat again unless the

workload and maintenance cause are corrected.

11. Autovacuum Tuning

```
ALTER TABLE app.orders SET (  
    autovacuum_enabled = true,  
    autovacuum_vacuum_scale_factor = 0.02,  
    autovacuum_vacuum_threshold = 5000,  
    autovacuum_analyze_scale_factor = 0.01,  
    autovacuum_analyze_threshold = 5000  
);  
  
SELECT relname, reloptions  
FROM pg_class WHERE oid='app.orders'::regclass;
```

Setting area	Tuning principle
Scale factors	Large tables often need smaller values because percentages become large row counts.
Base/max thresholds	Use base floor and PG18 maximum threshold to cap delayed triggering where appropriate.
Workers/slots	Increasing workers needs CPU, memory, I/O, and PG18 worker-slot review.
Cost delay/limit	Balance catch-up speed against foreground latency; anti-wraparound failsafe can override delays.
Memory	autovacuum_work_mem is per worker; larger values may reduce repeated index cycles but multiply.
Insert vacuum	Important for append-heavy tables to maintain visibility and freezing.
Freeze ages	Tune from XID rate and table behavior; do not use as a substitute for capacity.
Logging	Enable useful log_autovacuum_min_duration to see duration, skips, and outcomes.

Validation Change one evidence-backed parameter set at a time, observe several maintenance cycles, and compare duration, worker saturation, dead-tuple trend, foreground latency, and XID age.

12. VACUUM Options and Special Cases

Option/case	Operational note
ANALYZE	Runs ANALYZE after vacuum; planner statistics purpose differs from dead-tuple cleanup.
FREEZE	Aggressively freezes eligible tuples; not a universal routine requirement.
PARALLEL n	Parallelizes supported index vacuum/cleanup work, not the heap scan or ANALYZE.
SKIP_LOCKED	Skips relations that cannot be locked immediately, but index/child access can still block.
INDEX_CLEANUP	AUTO normally decides; OFF can accelerate urgent protection but deferred cleanup has cost.
TRUNCATE	Attempts to return empty tail pages; may need stronger lock briefly and can be disabled.
BUFFER_USAGE_LIMIT	Controls VACUUM/ANALYZE buffer strategy; too high can evict useful pages.
DISABLE_PAGE_SKIPPING	Only when visibility map is suspected corrupt due to hardware/software issue.
Temporary tables	Autovacuum cannot process them; session must maintain them.
Partitioned parent	Autovacuum processes partitions, not the storage-less parent; manually ANALYZE parent as needed.
VACUUM FULL progress	Reported in pg_stat_progress_cluster, not pg_stat_progress_vacuum.

13. Prevention and Production Hardening

- ☐ Alert on dead-tuple growth rate, not only a fixed count.
- ☐ Track heap/index/total size trends and disk headroom.
- ☐ Monitor autovacuum duration, failures, skips, worker saturation, and progress.
- ☐ Enforce transaction timeouts where application semantics allow.
- ☐ Assign owners and monitoring to every replication slot and prepared transaction.
- ☐ Use per-table settings for large/high-churn objects.

- ☐ Track XID and MultiXact age independently of ordinary bloat.
- ☐ Keep planner statistics current, including partitioned parents.
- ☐ Test recovery commands and privileges in nonproduction.
- ☐ Review write amplification and unused indexes that make vacuum more expensive.

14. Common Mistakes and Corrections

Mistake	Correct view
VACUUM deletes old rows immediately.	It removes versions only after no relevant snapshot needs them.
VACUUM always shrinks files.	Plain VACUUM mainly makes internal space reusable; tail truncation is limited.
n_dead_tup equals exact bloat.	It is an estimate and not a byte-accurate bloat measure.
VACUUM FULL is routine maintenance.	It is a rewrite with ACCESS EXCLUSIVE and extra disk requirements.
Autovacuum off means no vacuum ever.	Anti-wraparound vacuum can still launch, but routine disabling is dangerous.
ANALYZE and VACUUM are the same.	They serve related but different cleanup and planner-statistics roles.
More workers always fixes lag.	Workers compete for CPU, memory, I/O, and locks.
Higher work memory is always safe.	Memory applies per worker and can multiply.
Parallel VACUUM parallelizes everything.	It applies to supported index phases, not heap scan/ANALYZE.
A running VACUUM must be working effectively.	A pinned horizon can prevent expected tuple removal; inspect blockers and results.

15. Production Checklist

- ☐ Disk headroom and backups are verified.
- ☐ Top table/index growth and dead-tuple trends are known.
- ☐ Last vacuum/analyze and per-table reloptions are reviewed.

- ☐ Running vacuum phase and progress are checked.
- ☐ Long transactions, prepared xacts, slots, and standby feedback are reviewed.
- ☐ Locks, workers, memory, CPU, and I/O are checked.
- ☐ Plain targeted VACUUM is preferred for catch-up.
- ☐ VACUUM FULL/rewrite has separate downtime and space approval.
- ☐ Per-table tuning is tested with measured thresholds.
- ☐ XID and MultiXact ages are monitored.
- ☐ Post-change maintenance and query metrics are validated.

16. Official-Documentation Validation

Concept	Validated result
MVCC leaves obsolete tuple versions	Correct.
VACUUM reclaims dead-tuple storage	Correct; plain VACUUM usually makes it reusable internally.
VACUUM always returns space to OS	Incorrect.
Autovacuum uses threshold plus scale factor	Correct; PostgreSQL 18 also includes maximum and insert thresholds.
VACUUM updates planner statistics fully	Incomplete; ANALYZE collects planner distribution statistics.
Long transactions can block cleanup	Correct, as can prepared transactions and slot horizons.
Parallel VACUUM scans heap in parallel	Incorrect; parallel workers are used for supported index phases.
VACUUM FULL is best when lagging	Incorrect as a general response.
Visibility map helps index-only scans	Correct, subject to page visibility state.
Falling behind can lead to wraparound risk	Correct when freezing/age protection also falls behind.

17. References

- PostgreSQL 18 - Routine Vacuuming: <https://www.postgresql.org/docs/18/routine-vacuuming.html>
- PostgreSQL 18 - VACUUM: <https://www.postgresql.org/docs/18/sql-vacuum.html>
- PostgreSQL 18 - Vacuum Configuration: <https://www.postgresql.org/docs/18/runtime-config-vacuum.html>
- PostgreSQL 18 - VACUUM Progress: <https://www.postgresql.org/docs/18/progress-reporting.html#VACUUM-PROGRESS-REPORTING>
- PostgreSQL 18 - vacuumdb: <https://www.postgresql.org/docs/18/app-vacuumdb.html>
- PostgreSQL 18 - Resource Consumption: <https://www.postgresql.org/docs/18/runtime-config-resource.html>
- PostgreSQL 18 - pg_replication_slots: <https://www.postgresql.org/docs/18/view-pg-replication-slots.html>
- PostgreSQL 18 - pg_visibility: <https://www.postgresql.org/docs/18/pgvisibility.html>
- Source article - Kiran Sabne: <https://kiransabne.dev/postgresql-vacuum-why-it-exists-and-what-happens-when-it-falls-behind>

Validation basis: PostgreSQL 18 official documentation was used as primary authority. The linked article supplied the topic direction; inaccessible third-party wording was not treated as authoritative. Confirm documentation for the deployed major version and managed-service restrictions.